SUP DE VINCI

Ingénierie des Systèmes d'Information

TP1 : Intelligence Artificielle et Cybersécurité

AI-Driven Malware : Keylogger

Contributeurs :

Josiane EGLA
Samuel MENSAH

Champ	Détail
Matière	IA et Cybersécurité
Année	2025-2026
Type de rendu	Rapport de TP - Binôme
Parties traitées	I à V (obligatoire + extensions B et C + dashboard)
Lien github	https://github.com/Ecknard/keylogger_mlg.git

Sommaire

Ce rapport couvre les quatre parties obligatoires du TP ainsi que les extensions implémentées :

- Partie I : Le keylogger de base
- Partie II : Analyse de sentiments et détection d'anomalies
- Partie III : Classification de données sensibles
- Partie IV : Génération de rapports et visualisation
- Partie V : Extensions (contexte applicatif, chiffrement, dashboard)
- Architecture globale du projet
- Réflexion éthique et conclusion

Partie I : Le keylogger de base

Tâche 1 : Analyse conceptuelle du keylogger

Question 1.1 : Définition et dangers

Un keylogger (de l'anglais key = touche et logger = enregistreur) est un outil logiciel conçu pour surveiller et enregistrer, en temps réel et de manière transparente pour l'utilisateur, toutes les frappes effectuées sur un clavier. Il capture ainsi l'intégralité de ce qui est saisi (mots de passe, messages, numéros de carte bancaire, identifiants professionnels) et stocke ces informations localement ou les transmet à un tiers via le réseau.

Il existe deux familles principales de keyloggers :

- Les keyloggers logiciels, qui s'installent comme des programmes sur la machine. Ils peuvent fonctionner par hooking d'API (en détournant des fonctions système comme GetAsyncKeyState sur Windows), par injection dans des processus légitimes, ou en opérant au niveau du noyau (kernel-level) pour échapper aux antivirus.
- Les keyloggers matériels, qui prennent la forme d'un petit boîtier USB inséré entre le clavier et l'ordinateur. Ils ne laissent aucune trace logicielle, ce qui les rend particulièrement difficiles à détecter sans inspection physique.

Dans ce TP, la bibliothèque Python pynput est utilisée pour enregistrer un callback système appelé automatiquement à chaque événement clavier. Le tableau ci-dessous présente les quatre dangers concrets auxquels un utilisateur infecté est exposé.

Danger	Ce qui se passe concrètement	Conséquence pour la victime
Vol des identifiants	Chaque mot de passe saisi (banque, messagerie, VPN) est capturé en clair au moment de la saisie	Piratage de comptes, usurpation d'identité, accès aux données bancaires
Espionnage industriel	Courriels, rapports, stratégies et code source sont enregistrés puis transmis à un attaquant	Perte d'avantage concurrentiel, préjudice financier et atteinte à la réputation
Persistance longue	Le keylogger survit aux redémarrages via des clés de registre ou des tâches planifiées, sans symptôme visible	Des semaines de données peuvent être exfiltrées avant détection, les dégâts sont alors considérables
Exfiltration automatisée	Les logs sont envoyés périodiquement à un serveur C&C via SMTP, HTTP/S ou webhooks	L'attaquant exploite les données en temps quasi-réel sans accès physique à la machine

Question 1.2 : Usages légitimes

Si le keylogger est souvent associé à des activités malveillantes, il existe des usages tout à fait légaux, à condition que certaines obligations réglementaires soient strictement respectées. En France, la Loi Godfrain (art. 323-1 du Code pénal) et le RGPD encadrent strictement ces pratiques : toute utilisation sans consentement explicite est passible de deux ans d'emprisonnement et de 60 000 euros d'amende.

Secteur	Usage potentiel	Condition légale à respecter
Entreprise / DSI	Audit de sécurité interne, investigation post-incident, détection d'une menace interne	Consentement écrit des employés, mention dans la charte informatique, conformité RGPD (art. 5, 6 et 13)
Contrôle parental	Supervision des activités en ligne d'un enfant mineur pour le protéger des contenus inappropriés	Autorité parentale légale sur un mineur ; des solutions moins invasives sont généralement préférables
Forensic / SOC	Reconstitution des actions d'un attaquant lors d'une investigation judiciaire	Mandat judiciaire ou autorisation hiérarchique documentée, chaîne de preuve intègre
Pentest / Red Team	Simulation d'attaque keylogger dans le cadre d'un test d'intrusion commandé	Contrat de prestation signé, périmètre délimité, autorisation écrite du RSSI et de la direction

Tâche 2 : Installation et configuration de l'environnement

Pour isoler le projet et ses dépendances, nous avons créé un environnement virtuel Python dédié. Les commandes suivantes permettent de le configurer :

```
python -m venv env
.\env\Scripts\activate          # Windows
source env/bin/activate           # Linux / macOS
pip install -r requirements.txt
python -c "import pynput; print(pynput.__version__)"
```

Toutes les dépendances sont consignées dans requirements.txt. Notre projet s'appuie sur pynput (capture clavier), scikit-learn et joblib (modèles ML), langdetect (détection de langue), plotly et streamlit (visualisation), ainsi que cryptography (AES-256-GCM pour l'extension C).

Tâche 3 : Capture basique des frappes (keylogger.py)

Question 3.1 : La fonction processkeys(key)

La fonction processkeys() est le cœur du keylogger. pynput l'appelle automatiquement à chaque appui de touche et lui passe un objet qui peut être soit un KeyCode (touche alphanumérique, possède l'attribut .char), soit un Key (touche spéciale comme Ctrl ou F1, sans attribut .char). Notre implémentation gère les deux cas via un bloc try/except AttributeError et enrichit simultanément le buffer de texte ainsi que la liste keystroke_metadata (timestamp, inter_key_delay, key_type, char).

Question 3.2 : Le Listener et le paramètre on_press

Le Listener est l'objet pynput qui démarre un thread système d'écoute des événements clavier. Le paramètre on_press attend une référence à une fonction (sans parenthèses), laquelle sera appelée automatiquement à chaque appui. Nous lui passons donc on_press=processkeys. Ce pattern s'appelle un callback : c'est pynput qui décide du moment d'appel, et non le code principal.

Question 3.3 : Démarrer l'écoute (with et join)

Instruction	Rôle et comportement
with keyboard_listener:	Le gestionnaire de contexte appelle start() à l'entrée et stop() à la sortie, même si une exception est levée. Cette approche est plus propre qu'un try/finally manuel.
keyboard_listener.join()	Cette instruction bloque le thread principal en attente que le listener se termine. Sans join(), Python passerait à la suite et quitterait le programme avant toute capture.
Comportement global	Une fois lancé, le programme attend indéfiniment les frappes et appelle processkeys() pour chacune. L'arrêt se fait avec Ctrl+C.

Tâche 4 : Amélioration de l'affichage (variable log)

Question 4.1 : Variable globale log

La variable log est une chaîne de caractères (str) initialisée à la chaîne vide qui sert de buffer temporaire. Elle accumule les frappes entre deux sauvegardes. La fonction processkeys() distingue cinq cas : caractère alphanumérique (key.char concaténé, avec correctif AZERTY), espace (ajout de ' '), entrée (ajout de '\n'), backspace (suppression du dernier caractère) et tabulation (ajout de '\t'). Un verrou threading.Lock protège le buffer lors des accès concurrents entre le thread de capture et le thread de flush.

Question 4.2 : Pourquoi un bloc try / except ?

Lorsque l'utilisateur appuie sur une touche spéciale (F1, Ctrl, Alt, Shift), pynput passe un objet de type Key qui ne possède pas d'attribut. char. Si l'on tente d'y accéder, Python lève une AttributeError. Le bloc try/except AttributeError permet donc de distinguer les deux types de touches et d'adapter le traitement sans faire planter le programme.

Tâche 5 : Persistance dans un fichier log

Question 5.1 : Bibliothèques supplémentaires

Module	Utilité dans ce projet	Exemple d'usage

pathlib	Chemins portables et absolus indépendants du répertoire courant	ROOT = Path(__file__).resolve().parent ; DATA = ROOT / 'data'
threading	Flush périodique sans bloquer la capture, analyse IA en arrière-plan	threading.Timer(10, report) et threading.Lock()
datetime	Horodater chaque sauvegarde dans le fichier log et les événements	datetime.now().strftime('%Y-%m-%d %H:%M:%S')

Question 5.2 : Fonction report()

Élément	Explication
open(LOG_PATH, 'a')	Le mode 'a' (append) ajoute à la fin du fichier sans écraser son contenu. Le mode 'w' (write) réinitialiserait le fichier à chaque appel, ce qui ferait perdre tous les logs précédents.
Capture atomique avec lock	Sous _buffer_lock, nous copions log et keystroke_metadata dans des variables locales puis vidons les globales. L'écriture disque et l'analyse IA se font ensuite hors du verrou.
with open(...)	Le gestionnaire de contexte garantit la fermeture du fichier et le flush du buffer système, même en cas d'exception.
global log, keystroke_metadata	Sans cette déclaration, Python créerait des variables locales lors de la réinitialisation. Les globales ne seraient alors jamais vidées et les données seraient dupliquées.

Question 5.3 : Appel périodique avec threading.Timer

L'approche retenue est celle d'un timer non bloquant qui appelle report() toutes les 10 secondes. À la fin de chaque exécution, la fonction se relance elle-même via threading.Timer (avec timer.daemon = True pour un arrêt propre). Le flush déclenche en parallèle deux threads daemon supplémentaires : l'un écrit les métadonnées dans metadata.json, l'autre lance l'analyse IA (sentiment et détection de données sensibles) sur le texte qui vient d'être capturé.

Question 5.4 : Points faibles du keylogger de base

Dimension	Point faible identifié	Piste d'amélioration
Détection AV	pynput possède une signature connue des antivirus, détectable à l'analyse statique	Obfuscation, hooking bas-niveau, compilation en bytecode
Lisibilité	Le fichier log.txt brut est difficile à analyser, sans structure ni contexte temporel	JSON horodaté avec métadonnées (Partie II)
Contexte applicatif	Impossible de savoir dans quelle application les frappes ont été effectuées	Extension B : win32gui/xdotool/AppKit pour capturer l'application active
Couverture layout	Risque de saisie mal interprétée selon la disposition clavier	Correctif AZERTY basé sur key.vk (implémenté en v2)
Sécurité du fichier	log.txt est stocké en clair et lisible par quiconque a accès à la machine	Chiffrement AES-256-GCM du fichier (Extension C)

Partie II : Analyse de sentiments et détection d'anomalies

Tâche 6 : Analyse de sentiments (sentiment_analyzer.py)

Question 6.1 : Choix de la bibliothèque

Après comparaison des options disponibles, nous avons retenu une approche ML sans lexique : TF-IDF sur n-grammes de caractères combiné à une régression logistique (scikit-learn). Ce choix s'écarte volontairement de VADER, limité à l'anglais, afin de couvrir directement cinq langues (français, anglais, espagnol, allemand, italien) via les patterns morphologiques partagés, par exemple 'horr' capturé dans horrible / horribile / horroroso. Le modèle s'entraîne en moins d'une seconde au premier lancement et est mis en cache dans data/sentiment_model.joblib.

Approche	Points forts	Limitations	Verdict
TF-IDF char 2-5 + LogReg	Multilingue sans lexique, entraînement rapide, modèle léger	Corpus à maintenir (500+ exemples équilibrés)	Retenu
VADER	Textes courts et informels, aucun entraînement requis	Anglais uniquement, pas adapté à notre cas multilingue	Écarté
TextBlob	Simple, plusieurs langues via traduction	Moins précis sur l'argot, dépendance traduction	Alternative
Transformers HuggingFace	État de l'art, multilingue, très précis	Lourd, requiert idéalement un GPU	Surdimensionné

Question 6.2 : Implémentation du module de sentiment

Le pipeline sklearn combine un TfidfVectorizer (analyzer='char_wb', ngram_range=(2,5), sublinear_tf=True) et une LogisticRegression (C=3.0, class_weight='balanced'). La fonction analyze_sentiment() retourne un dictionnaire enrichi : score continu dans l'intervalle [-1, +1] calculé par tanh(logit(p)), label sur cinq niveaux (très_positif, positif, neutre, négatif, très_négatif), langue détectée via langdetect avec drapeau emoji, et score de confiance pondéré. Si le texte comporte moins de deux mots, le label 'trop_court' est retourné sans calcul. Le singleton _get_model() évite de recharger le modèle à chaque appel.

Structure du fichier sentiments.json généré :

```
{ "timestamp": "2026-04-20T14:32:01", "text": "...",
  "sentiment": "positif", "score": 0.6369,
  "details": { "neg": 0.0, "neu": 0.446, "pos": 0.554, "compound": 0.6369 },
  "word_count": 8 }
```

Question 6.3 : Intégration dans la boucle de capture

La fonction analyze_sentences_from_log() découpe le texte du buffer par retours à la ligne, puis par ponctuation forte (. ! ? ;), et appelle analyze_sentiment() sur chaque segment valide. L'appel est orchestré par _run_analysis() dans report(), exécuté dans un thread daemon afin de ne pas bloquer la capture clavier. Les résultats sont stockés dans data/sentiments.json en mode append, avec les enregistrements trop_court filtrés.

Tâche 7 : Détection d'anomalies (anomaly_detector.py)

Question 7.1 : Collecte des métadonnées de frappe

Chaque frappe est enrichie de quatre métadonnées comportementales, stockées dans la liste globale keystroke_metadata puis périodiquement sauvegardées dans data/metadata.json (avec rétention des 10 000 dernières entrées).

Métadonnée	Description	Calcul
timestamp	Horodatage précis de chaque frappe (format Unix epoch et ISO 8601)	time.time() et datetime.fromtimestamp(now).isoformat()
inter_key_delay	Délai en secondes entre deux frappes consécutives	round(now - last_key_time, 4)
key_type	Catégorie : alphanum, special, navigation, modifier, function	Fonction _classify_key_type() qui teste l'objet key
char	Caractère enregistré dans le log (ou étiquette [BACK], '\t', etc.)	Valeur retournée par processkeys() après correctif AZERTY

Question 7.2 : Modèle de détection d'anomalies

Notre choix s'est porté sur Isolation Forest (sklearn.ensemble.IsolationForest avec n_estimators=100, contamination=0.05, random_state=42, n_jobs=-1). C'est un algorithme non supervisé, ce qui est adapté car il n'existe pas d'exemples de frappe anormale étiquetés. Huit features sont extraites par fenêtre glissante de 20 frappes : mean_delay, std_delay, max_delay, min_delay, alphanum_ratio, special_ratio, burst_count normalisé et median_delay.

Question du TP	Réponse apportée
Combien de données pour l'entraînement ?	Un minimum de 100 événements est fixé (MIN_SAMPLES_TRAIN). En pratique, 30 à 60 minutes de frappe normale donnent un profil fiable ; au-delà, les gains sont marginaux.
Pourquoi normaliser ?	Les délais (0,12 s) et les ratios (0 à 1) ont des échelles très différentes. Sans StandardScaler, les features à forte variance domineraient le calcul de distance.
Contamination d'Isolation Forest ?	contamination=0.05 (soit 5 %) est retenue comme point de départ. Une valeur trop élevée produit de faux positifs, tandis qu'une valeur trop faible fait rater de vraies anomalies.
Sauvegarde du modèle ?	joblib.dump(model, MODEL_PATH) ; idem pour le scaler. La fonction load_model() recharge le modèle depuis le disque au démarrage.

Question 7.3 : Alertes en temps réel

La classe AnomalyMonitor s'exécute dans un thread daemon et vérifie la fenêtre des 20 dernières frappes toutes les 5 secondes (paramètre check_interval). La méthode train_if_ready() déclenche l'entraînement automatiquement dès que MIN_SAMPLES_TRAIN événements sont atteints. Si Isolation Forest prédit -1 (anomalie), une alerte horodatée est enregistrée dans data/alerts.json. Voici un exemple de structure :

```
{ "timestamp": "2026-04-20T15:12:03", "type": "keystroke_anomaly",
  "is_anomaly": true, "score": -0.4231, "window_size": 20 }
```

Trois canaux d'alerte sont prévus : un fichier JSON (simple, retenu par défaut), une notification système via plyer (complexité moyenne), et un webhook Slack ou Teams via requests (complexité avancée). Seule la journalisation JSON est implémentée dans cette version.

Partie III : Classification de données sensibles

Tâche 8 : Détection par expressions régulières et modèle ML (sensitive_detector.py)

Question 8.1 : Patterns à détecter

Notre module implémente huit patterns regex qui couvrent les formats de données sensibles les plus courants, avec validation post-détection (algorithme de Luhn pour les cartes bancaires, zéro faux positif). Pour les mots de passe, qui ne suivent aucun format fixe, un classificateur ML prend le relais (voir question 8.2). Une déduplication par plage d'indices évite les doublons entre regex et détections ML.

Type de donnée	Format typique	Pattern implémenté
Email	user@domain.com (sous-domaines, TLD longs)	\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b
Téléphone FR (01 à 09)	0612345678, 06.12.34.56.78, +33612345678	(?<!\d)(?:\+33|0033)?[1-9]... (tous préfixes 0X)
Carte bancaire + Luhn	XXXX XXXX XXXX XXXX (16 chiffres)	\b(?:\d{4}[\s-]){3}\d{4}\b + _luhn_check()
Numéro de sécu FR	1 85 12 75 123 456 78 (15 chiffres)	\b[12]\s?\d{2}\s?\d{2}\s?\d{2,3}\s?\d{3}\s?\d{3}\s?\d{2}\b
IBAN FR	FR76 XXXX XXXX XXXX XXXX XXXX XXX	\bFR\d{2}[\s]?(?:\d{4}[\s]?){5}\d{3}\b
Adresse IPv4	192.168.1.1 (valeurs 0-255 validées)	(?:25[0-5]|2[0-4]\d|[01]?\d\d?) répété 4 fois
Token JWT	eyJxxx.yyyy.zzzz (JSON Web Token)	\beyJ[A-Za-z0-9_-]{10,}\.{...}\.{...}\b
Clé API / Bearer	api_key=abc123..., Authorization: Bearer ...	(?:Bearer|api[_-]?key|token|secret)[:\s=]+[A-Za-z0-9_.-]{16,}

Question 8.2 : Modèle de classification ML pour les mots de passe

Les mots de passe ne suivent aucun format fixe : une regex trop permissive génère de faux positifs et une regex trop stricte en manque. Nous avons donc entraîné un Random Forest (n_estimators=100) sur huit features extraites par extract_string_features() : longueur, entropie de Shannon, ratio de majuscules, ratio de chiffres, ratio de caractères spéciaux, indicateur longueur supérieure à 8, indicateur de présence simultanée des trois catégories (majuscule, chiffre et spécial), et ratio de caractères uniques.

Le choix du Random Forest face à Naive Bayes ou SVM se justifie ainsi : Naive Bayes suppose l'indépendance statistique des features, ce qui est faux ici (longueur et entropie sont corrélées). SVM demande un scaling très rigoureux et se révèle moins interprétable. Le Random Forest, quant à lui, gère naturellement les features mixtes et permet d'analyser les feature_importances_ pour comprendre quels critères pèsent le plus.

La métrique prioritaire est le rappel (recall) : un faux négatif (mot de passe non détecté) est bien plus grave qu'un faux positif. Nous avons donc utilisé class_weight={0:1, 1:2} afin de pénaliser davantage les faux négatifs lors de l'entraînement. Le rapport classification_report de sklearn est affiché lors de l'entraînement pour validation.

Question 8.3 : Masquage et protection avant écriture sur disque

Nous avons implémenté deux stratégies de masquage. La fonction mask_sensitive() remplace intégralement par des étoiles (sécurité maximale, perte d'information). La fonction redact_sensitive() applique une redaction intelligente qui conserve un format lisible : alice@example.com devient a****@***.com, la carte 4532 1234 5678 9012 devient **** **** **** 9012, et 06 12 34 56 78 devient ** ** ** ** 78. Pour ne pas décaler les indices lors du remplacement, les détections sont triées par position décroissante.

Approche	Bibliothèque	Ce qui est implémenté
Masquage complet (****)	re.sub() (bibliothèque standard)	mask_sensitive() : remplacement en place, irréversible
Redaction intelligente	re (bibliothèque standard)	redact_sensitive() : conserve le format (a****@***.com, **** 9012)
Hachage SHA-256	hashlib (bibliothèque standard)	hash_sensitive() : empreinte non réversible pour les logs d'audit
Chiffrement AES-256-GCM	cryptography	Extension C : chiffrement complet du fichier log

Partie IV : Génération de rapports et visualisation

Tâche 9 : Dashboard de visualisation (report_generator.py)

Question 9.1 : Choix des bibliothèques de visualisation

Nous avons retenu Plotly pour les graphiques et Jinja2 pour le templating HTML. Plotly offre des graphiques interactifs (zoom, survol, export PNG) qui s'intègrent directement dans le HTML via pio.to_html(), avec une API plus intuitive que celle de Matplotlib. Streamlit a été choisi pour le dashboard temps réel.

Bibliothèque	Type de visualisation	Utilisation
Plotly	Graphiques interactifs web (interactivité élevée)	Tous les graphiques du rapport HTML
Jinja2	Templates HTML (moteur de rendu)	Génération du rapport final data/report.html
Streamlit	Dashboard web complet en temps réel	Dashboard multilingue avec st.rerun()
Matplotlib	Graphiques statiques (aucune interactivité)	Non retenu

Question 9.2 : Les cinq visualisations implémentées

Visualisation	Source	Graphique et justification
Évolution des sentiments	sentiments.json	Line chart avec zones colorées (vert positif, gris neutre, rouge négatif) : les plages de stress ou d'énervement sont immédiatement visibles
Distribution des délais inter-touches	metadata.json	Histogramme avec courbe de densité : révèle le profil de frappe normal et les comportements anormaux (trop rapide = bot, longues pauses = interruptions)
Heatmap d'activité horaire	Timestamps des frappes	Heatmap heure x jour : identifie les plages d'activité intense et les périodes inhabituelles (par exemple un travail à 3 h du matin)
Répartition des données sensibles	detections.json	Donut chart : proportion de chaque type de donnée sensible capturée (email, carte, téléphone, mot de passe, etc.)
Timeline des anomalies	alerts.json	Scatter plot : chaque marqueur rouge représente une anomalie comportementale avec son score

Tâche 10 : Génération automatisée de rapports

Question 10.1 : Résumé textuel par NLP

Approche	Bibliothèque	Résultat généré
Mots-clés fréquents	collections.Counter + stopwords FR/EN	Top 10 des mots les plus saisis hors mots outils
Résumé statistique	Python natif	Pourcentage positif/négatif/neutre, nombre d'anomalies, données sensibles
LLM (API Claude)	API Anthropic via requests	Rapport narratif complet en langage naturel (optionnel)

Question 10.2 : Export du rapport

Nous avons choisi le format HTML généré via Jinja2 et Plotly. Les avantages principaux sont les suivants : les graphiques Plotly restent interactifs (zoomables, survolables), le fichier est partageable sans installation, et la génération est entièrement automatique grâce à la fonction generate_html_report() qui rend le template et écrit data/report.html.

Partie V : Extensions avancées

Nous avons implémenté trois extensions au-delà du minimum requis : l'Extension B (contexte applicatif), l'Extension C (chiffrement AES-256-GCM) et un dashboard Streamlit temps réel multilingue (extension/dashboard.py).

Extension B : Reconnaissance de contexte applicatif (app_context.py)

Cette extension enrichit chaque frappe avec le nom de l'application active au moment de la saisie. Cette information se révèle très utile pour distinguer les frappes effectuées dans un navigateur (données bancaires, mots de passe web) de celles effectuées dans un traitement de texte. Le module détecte automatiquement l'OS via platform.system() et applique la méthode adaptée, avec un fallback pygetwindow en cas d'échec.

OS	Bibliothèque utilisée	Informations récupérées
Windows	win32gui, win32process et psutil	Titre de la fenêtre active, nom du processus, PID
Linux (X11)	xdotool via subprocess	Titre de la fenêtre, PID du processus actif
macOS (Darwin)	AppKit.NSWorkspace via pyobjc	Nom localisé de l'application, Bundle ID, PID
Multi-OS (fallback)	pygetwindow	Titre de la fenêtre : abstraction cross-OS si les bibliothèques spécifiques sont indisponibles

Extension C : Chiffrement de bout en bout (encryption.py)

Le fichier log.txt stocké en clair constitue un point de faiblesse évident. Cette extension chiffre le fichier avec AES-256-GCM, qui offre à la fois le chiffrement et l'authentification intégrée : toute modification du fichier chiffré est détectée par la levée d'une InvalidTag lors du déchiffrement.

La dérivation de clé s'effectue via PBKDF2-SHA256 avec 480 000 itérations (recommandation OWASP 2023), ce qui permet de chiffrer le log à partir d'un mot de passe mémorisable plutôt que d'une clé brute à stocker. Les fonctions encrypt_file() et decrypt_file() gèrent le cycle complet, avec permissions restrictives 0o600 sur le fichier de clé (Unix).

```
key = os.urandom(32)                    # cle AES-256 aleatoire
aesgcm = AESGCM(key)
nonce = os.urandom(12)                  # IV aleatoire 96 bits
encrypted = aesgcm.encrypt(nonce, plaintext.encode(), None)
result = base64.b64encode(nonce + encrypted).decode()
key, salt = derive_key_from_password('MotDePasseSecret!')
```

Extension supplémentaire : Dashboard de supervision temps réel (dashboard.py)

Il s'agit d'un dashboard Streamlit doté d'un thème sombre de type cybersécurité (polices JetBrains Mono et Syne, palette #070b14). Il se rafraîchit via st.rerun() avec un compteur visible, lit directement les fichiers JSON à chaque cycle (zéro cache) et affiche les sentiments multilingues avec drapeaux emoji par phrase.

Composant	Technologie	Ce qui est affiché
En-tête et KPI	HTML/CSS personnalisé via st.markdown	Frappes totales, score moyen de sentiment, alertes de la dernière heure, données sensibles
Graphiques Plotly	Dark theme unifié (fond transparent, grilles sombres)	Timeline des sentiments, histogramme des délais, heatmap, carte des langues, radar de confiance
Table multilingue	st.dataframe avec drapeaux emoji	Phrases analysées avec langue détectée, label à 5 niveaux, score et confiance
Sidebar de contrôle	Sliders, selectbox et boutons Streamlit	Intervalle de rafraîchissement, vue active, avertissement keylogger inactif

Architecture globale du projet

Le projet suit l'architecture modulaire recommandée par le sujet. Chaque fichier a une responsabilité unique et les modules communiquent entre eux via des fichiers JSON stockés dans le répertoire data/. La structure finale est la suivante :

```
ai_keylogger_multilang/
  keylogger.py           <- Partie I   : capture, buffer thread-safe, pipeline IA
temps réel
  sentiment_analyzer.py  <- Partie II  : TF-IDF char + LogReg multilingue
(FR/EN/ES/DE/IT)
  anomaly_detector.py    <- Partie II  : Isolation Forest, AnomalyMonitor thread
daemon
  sensitive_detector.py  <- Partie III : 8 regex + Luhn + Random Forest, masquage
intelligent
  report_generator.py    <- Partie IV  : 5 graphiques Plotly, résumé NLP, rapport
HTML Jinja2
  extension/
    app_context.py       <- Extension B : contexte applicatif (Windows/Linux/macOS)
    encryption.py        <- Extension C : AES-256-GCM, PBKDF2 480k itérations
    dashboard.py         <- Dashboard Streamlit multilingue (st.rerun, drapeaux
emoji)
  data/                  <- logs, modèles ML, exports (gitignored)
    log.txt             metadata.json       sentiments.json
```

```
    alerts.json          detections.json     report.html
    sentiment_model.joblib   isolation_forest.joblib   sensitive_classifier.joblib
  tests/test_all.py      <- tests unitaires unittest (sentiment, regex, features,
crypto)
  requirements.txt       <- pynput, sklearn, langdetect, plotly, streamlit,
cryptography
  README.md
```

Flux de données entre modules (pipeline temps réel orchestré par report()) :

```
Clavier
  |
  v
keylogger.py processkeys()  -->  buffer log + keystroke_metadata (lock)
  |
  v  toutes les 10s : report() flush atomique
  +-->  data/log.txt                          (horodatage + contenu)
  +-->  data/metadata.json                    (append, max 10 000 entrées)
  +-->  Thread daemon : _run_analysis()
          +-->  sentiment_analyzer.py        -->  data/sentiments.json
          +-->  sensitive_detector.py        -->  data/detections.json (hachées)
  +-->  anomaly_detector.py (AnomalyMonitor) -->  data/alerts.json

report_generator.py (post-traitement)        -->  data/report.html
```

Réflexion éthique

Les tensions éthiques soulevées par ce TP

Ce TP soulève des questions importantes qui dépassent la dimension technique. Plusieurs réflexions s'en dégagent naturellement :

- La dualité de l'outil : exactement les mêmes techniques servent à protéger (détection des menaces internes) et à attaquer (vol de credentials). La frontière n'est pas technique ; elle est uniquement légale et contextuelle.
- L'IA amplifie le potentiel de nuisance : un simple keylogger qui enregistre du texte est déjà problématique, mais un keylogger qui classe automatiquement les mots de passe, détecte les données bancaires et analyse les émotions dans cinq langues représente un niveau de surveillance autrement plus intrusif. Plus nous intégrons de l'IA, plus la responsabilité du développeur est engagée.
- Le consentement change tout, mais modifie aussi le comportement : informer quelqu'un qu'un keylogger est actif est légalement requis, mais modifie radicalement sa façon de taper, ce qui réduit la pertinence de l'outil pour la sécurité offensive.
- Le risque de dérive : un outil déployé pour un usage légitime (audit de sécurité) peut facilement glisser vers une surveillance excessive si les accès et la durée de conservation ne sont pas strictement encadrés.

Mesures de protection intégrées (Privacy by Design)

Pour limiter les risques, plusieurs principes de Privacy by Design (RGPD) ont été appliqués dès la conception du projet :

Mesure	Implémentation dans le code
Masquage et redaction des données sensibles	mask_sensitive() et redact_sensitive() remplacent emails, cartes bancaires et numéros de sécu avant toute écriture sur disque
Hachage SHA-256 pour les audits	hash_sensitive() garantit que les valeurs détectées ne sont jamais stockées en clair

Chiffrement AES-256-GCM du fichier log	Extension C : log.txt peut être chiffré et n'est accessible qu'avec la clé secrète, avec authentification intégrée
Avertissement éthique dans le code	Chaque module porte un en-tête rappelant l'usage pédagogique et les sanctions légales (Loi Godfrain, RGPD)
Threads daemon pour arrêt propre	timer.daemon = True : le processus s'arrête proprement avec le programme principal, sans laisser de processus zombie

Conditions pour un usage légal

- Obtenir un consentement écrit et explicite avant tout déploiement.
- Informer clairement sur la nature des données collectées, la durée de conservation et les destinataires.
- Appliquer le principe de minimisation des données (RGPD art. 5c) : ne collecter que ce qui est strictement nécessaire.
- Restreindre l'accès aux logs aux seules personnes habilitées, avec journalisation des accès.

Conclusion

Ce TP nous a permis de construire un pipeline complet autour du keylogger : de la capture bas niveau des frappes jusqu'à la génération automatique de rapports, en passant par l'analyse de sentiment multilingue, la détection d'anomalies comportementales et la classification de données sensibles avec validation algorithmique (Luhn).

Sur le plan technique, la combinaison des trois approches de détection constitue l'aspect le plus intéressant : les regex pour les formats structurés (rapides et déterministes), le Random Forest pour les mots de passe (adaptatif, rappel privilégié) et l'Isolation Forest pour les anomalies de frappe (non supervisé, fenêtre glissante). Le moteur de sentiment TF-IDF char-n-gram + Logistic Regression constitue par ailleurs une alternative légère et élégante à VADER, fonctionnant sur cinq langues sans lexique.

Sur le plan éthique, ce travail met en évidence qu'une même technique peut servir ou nuire selon le contexte. En tant que futurs professionnels de la sécurité, c'est une responsabilité que nous devons garder présente à l'esprit dans chaque projet, dès la phase de conception.